

Sprint Week 1 – Summer 2026

Kick-Off: June 18th | Due: July 5th

Overview

In this sprint you will build two separate Java programs that communicate over HTTP:

- **A Server (REST API)** — built with Spring Boot, backed by a relational database
- **A Client (CLI App)** — a Java command-line program that connects to your API and answers four questions

You are welcome to pitch your own project idea, but it must meet all requirements below.

Part 1 — REST API (Server)

Your API must:

- Be built with **Spring Boot** and expose a REST API over HTTP
- Store data in a **relational database** (e.g. MySQL)
- Support all four HTTP verbs: **GET, POST, PUT, DELETE**
- Be fully testable using **Postman or another related tool**
- Implement the four entities and their relationships described below
- Live in its **own GitHub repository**, separate from the CLI
- Show clear evidence of a **branching and Pull Request (PR) workflow**
- Have a **GitHub Project board** set up and actively used for team collaboration

Part 2 — CLI App (Client)

Your CLI must:

- Be a separate **Java program** that connects to your API via HTTP
- Make exactly **4 GET requests** — one per question below (no POST/PUT/DELETE from the CLI required)
- Live in its **own GitHub repository**, separate from the API
- Have **GitHub Actions** configured to run tests automatically on PRs and merges to **main**
- Use **JUnit** for testing, including **mocking** — aim for strong test coverage

Data Model

Entity	Fields
City	id, name, state, population
Passenger	id, firstName, lastName, phoneNumber
Airport	id, name, code
Aircraft	id, type, airlineName, numberOfPassengers

Relationships:

- A **City** has many **Airports**
- An **Airport** belongs to one **City**
- A **Passenger** lives in one **City** and can fly on many **Aircraft**
- An **Aircraft** can carry many **Passengers** and operate out of many **Airports**

Note: Use AI to help you generate some of the example data if needed!

The Four Questions (CLI Must Answer These)

1	What airports are in each city?
2	What aircraft has each passenger flown on?
3	What airports does each aircraft take off from and land at?
4	What airports have passengers used?

CHALLENGE — Pagination & Sorting with JPA

This is a required part of the sprint. The goal isn't just to get it working. You need to understand *what* you implemented and *why*. Use AI tools to help you learn and explore but be prepared to explain every line in your own words.

What You Need to Implement

Add **pagination** and **sorting** support to at least **one of your existing API endpoints**, using Spring Data JPA's built-in **Pageable** support.

Your implementation should allow callers to control:

- How many results are returned per page (**size**)
- Which page they are on (**page**)
- What field results are sorted by (**sort**)
- Sort direction — ascending or descending

Reflection Questions

Once implemented, answer the following questions in your sprint reflection document. **Use specific examples from your own code and API** — generic or AI-generated answers will not be accepted.

Q1. Which two endpoints did you add pagination to, and why did those make sense to paginate?

e.g. "We paginated GET /passengers because in a real system there could be thousands of records..."

Q2. Show a Postman request that uses pagination and sorting (Show a screenshot from postman when you test) on one of your endpoints. What URL did you use, and what did the response look like?

Q3. In your own code, what does the Pageable parameter do in your repository or controller? Paste the relevant method signature and explain it.

Q4. What is the difference between Page<T> and List<T> as a return type? Why might you choose one over the other?

A note on using AI: You're encouraged to use AI to help you understand pagination and JPA — ask it to explain concepts, help you debug, or walk you through the Spring docs. But the reflection answers must come from *you*. If you can't explain your own code without AI assistance, that's a signal that you should be taking time to really study and understand what it is giving you

Demo Video

Each team must submit a short demo video where:

- **All team members** speak and participate
- You **showcase the working features** of your project
- You discuss honestly — what went well, what didn't, and what you'd do differently

Submission Requirements

Submit the following to the portal:

- GitHub link for your **API repo**
- GitHub link for your **CLI repo**
- Link to your **demo video (Make sure we have access!)**
- A short-written self-reflection — highlights, challenges, and lessons learned

Resources

- **Jordan Example Repos:** <https://github.com/JKells99?tab=repositories>
- **Jamie Example Repos:** <https://github.com/jamie-keyin>
- **Spring Boot Docs:** <https://docs.spring.io/spring-boot/index.html>
- **Mock Client Example from Class:** https://github.com/jamie-keyin/S4_2023_Spring_HTTP_Client_Mock

Grading — Pass Outstanding Criteria

To achieve a Pass **Outstanding** grade your project must demonstrate:

- Goes **above and beyond** in some meaningful way
- **Two separate repos** — one for the API, one for the CLI
- **Strong test coverage** on the CLI with a substantial number of tests written
- **GitHub Actions passing** on the CLI repo
- **Strong evidence of team collaboration:** regular commits from all members, consistent branching/PR workflow, and an active project board
- **All four questions** answered correctly from the CLI